Submission Catalogue

Submission Catalogue **1**
1. Overview **2**
2. UML Class Diagram **4**
3. UML Object Diagrams **6**
Scenario 1: Player Uses an Item to Defeat a Monster 6
Scenario 2: Player Manages Inventory Weight to Progress 8
4. UML Sequence Diagrams **10**
Scenario 1: Player Uses an Item to Defeat a Monster 10
Scenario 2: Player Manages Inventory Weight to Progress 12

1. Overview

The system implements a data-driven adventure game engine following the **Model-View-Controller (MVC)** pattern. The engine loads game content from external JSON files at runtime via **JsonGameLoader**, enabling flexible game creation without modifying source code. It enforces a strict separation between game logic **(Model)**, user interaction **(two interchangeable Controllers)**, graphical presentation **(View)**, and application bootstrapping **(Engine Driver)**.

The **Model** layer encapsulates all game state and logic. **IGameModel** defines the contract between the Controllers and the Model — neither controller accesses internal classes such as **Player**, **Room**, or **Item** directly. **GameModel** orchestrates movement, inventory, puzzle solving, monster encounters, and scoring. Action methods return String messages; read-only query methods (getInventoryItemNames, getRoomItemNames, getExaminableNames, getItemImage, getCurrentRoomImage, getGameName) serve the graphics layer, keeping the Model UI-agnostic and free of I/O.

The **Player** tracks health (0–100), score, current room number, and an **Inventory** with a weight limit of 13. Health is categorized by the **HealthStatus** enum (AWAKE, FATIGUED, WOOZY, SLEEP); the game ends when health reaches zero. The **GameWorld** serves as a centralized registry for all **Room**, **Item**, **Fixture**, **Puzzle**, and **Monster** objects. Rooms are connected via directional exits modeled by the **Direction** enum (positive = passable, zero = wall, negative = blocked by an active puzzle or monster). Items have limited uses and weight. Puzzles gate blocked exits and are solved via text answers or item usage. Monsters block exits and deal damage on movement attempts, defeated by using the correct item. Solving a puzzle or defeating a monster deactivates it, opens blocked exits, and awards score. Fixtures are immovable objects examined for clues. Each **Item**, **Fixture**, **Puzzle**, **Monster**, and **Room** may carry an optional picture; the Model selects the image to surface for the current room (active monster → active puzzle → room default).

The engine offers two interchangeable controllers behind **IGameModel**: **GameController** drives the console loop — reading input from a **Readable**, dispatching commands, and writing to an **Appendable** (supporting move, look, take/drop/use/examine, answer, save/restore). **GraphicsController** drives the Swing GUI: it implements **ViewListener** so the View can push user events to it, and it calls **IGameView** methods to refresh the room image, description, navigation buttons, health status, and inventory after each model change; blocked-move feedback and game-over detection are handled centrally.

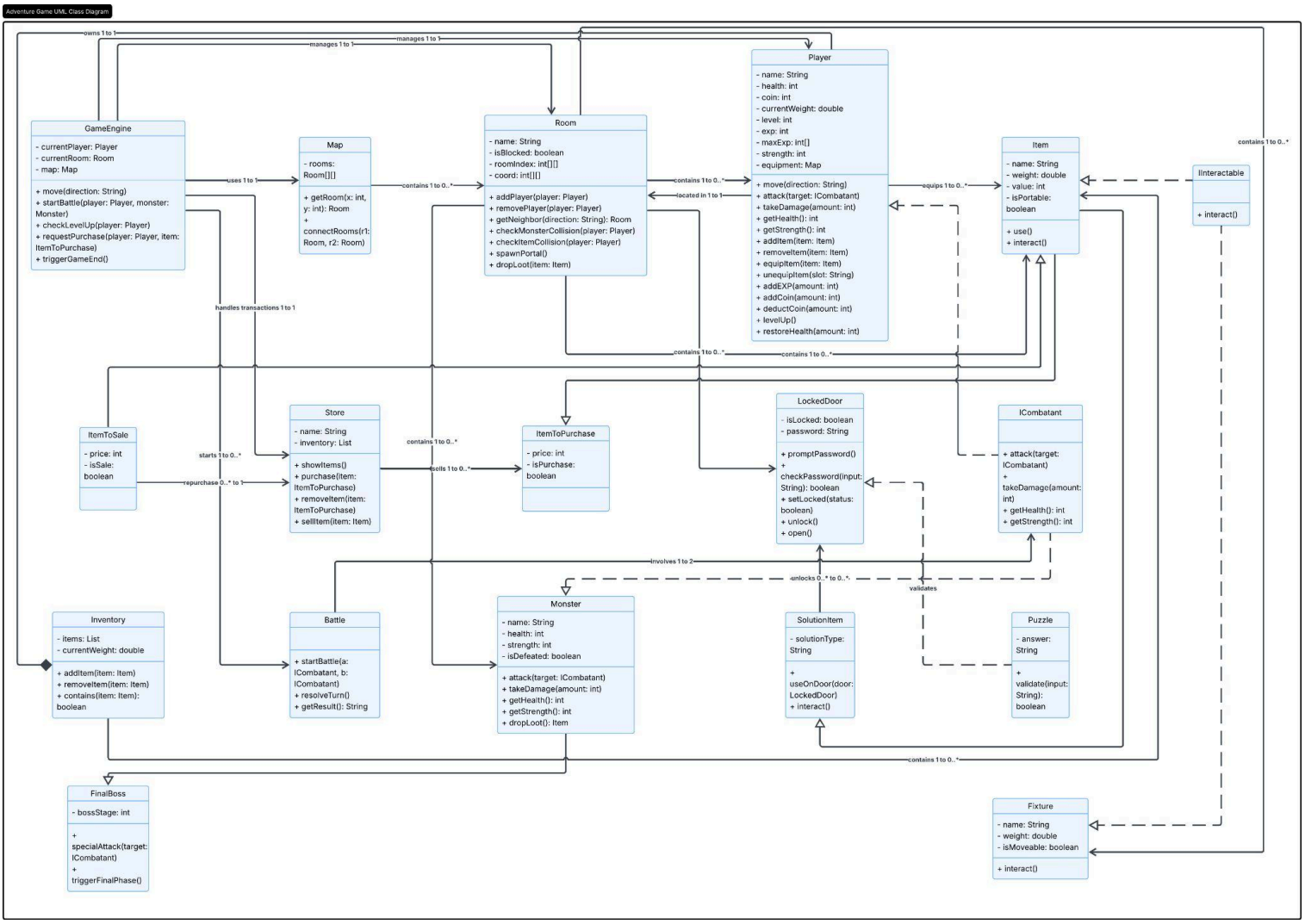
The View layer is defined by **IGameView** and implemented by MainFrame, a **JFrame** assembled from four quadrant panels — **ViewPanel** (room image), **NavigationPanel** (directional and action buttons), **DescriptionPanel** (room narrative), and **InventoryPanel** (items and status) — plus a File menu (About / Save / Restore / Exit). Supporting classes

(**ModalDialogs**, **ImageUtils**, **RoundedButton**, **TeamUiTheme**, **GradientOutlineBorder**) provide reusable dialogs, image loading, and a consistent theme. The View holds no game state: gestures are forwarded through **ViewListener**, updates arrive through **IGameView**.

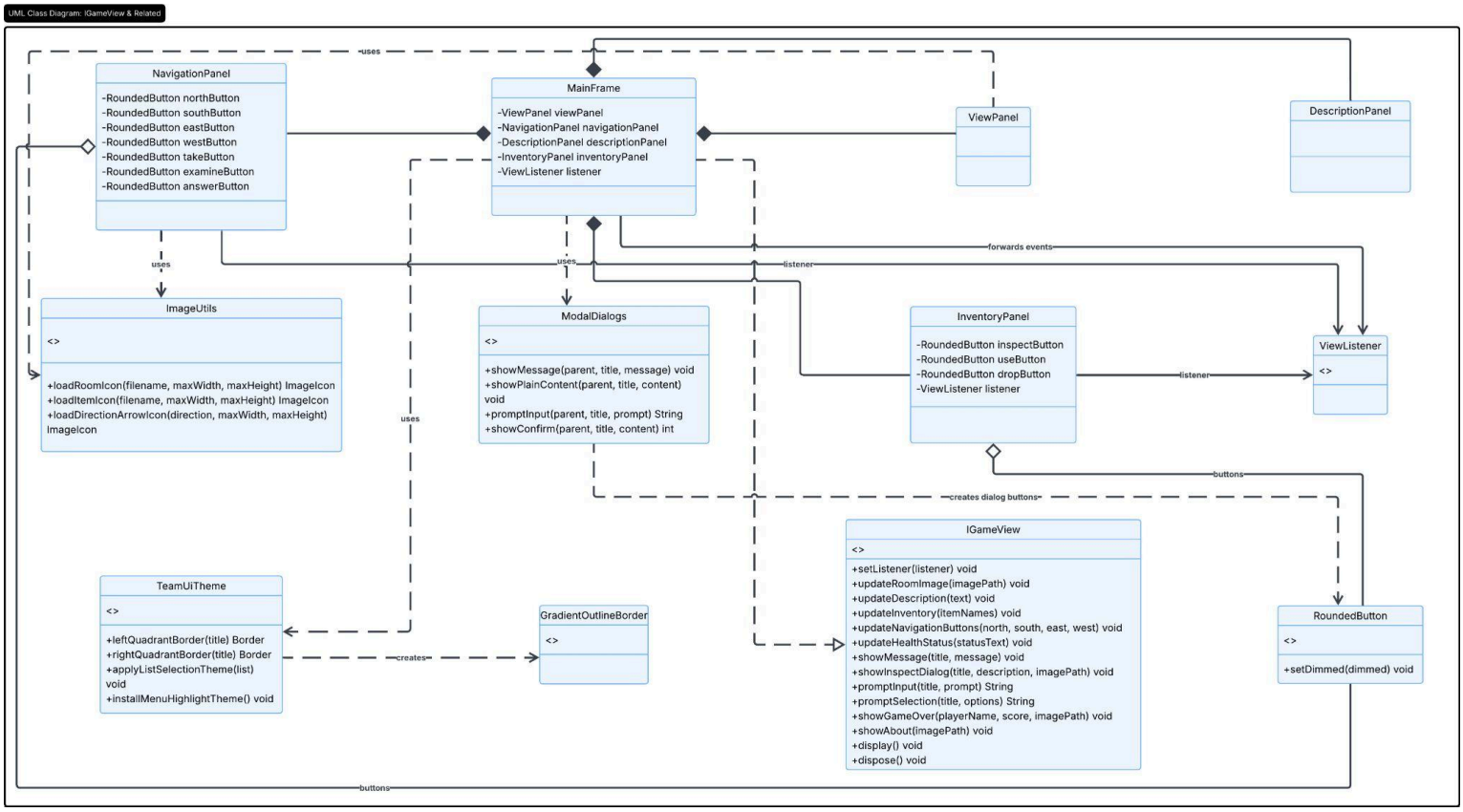
GameSaveManager persists player stats, inventory, item usage, puzzle/monster states, and room exits to JSON, and is shared by both controllers. **GameEngineApp** selects the run mode from the command line: **-text** for console play, **-graphics** for the Swing GUI (launched on the EDT via **SwingUtilities.invokeLater**), and **-batch <source> [target]** for scripted runs that read commands from a file and optionally write output to another file.

Overall, the design enforces strict separation of concerns: the Model owns all logic, the Controllers handle interaction (text or graphical), the View is purely presentational, and the data-driven architecture keeps game content independent of engine code — each layer can be tested and maintained in isolation, and swapping the text UI for the Swing UI requires no changes to the Model.

2. UML Class Diagram



This diagram provides a complete overview of all classes and relationships in the system. This diagram provides a complete overview of all classes and relationships in the system.



This GUI class diagram describes the structure of the Swing-based view layer and how its components collaborate.

3. UML Object Diagrams

Scenario 1: Player Uses an Item to Defeat a Monster

Context: The Player has entered the Foyer but cannot proceed East because the path is blocked by the Teddy Bear monster. The player must use the Hair Clippers item to defeat it and unlock the exit.

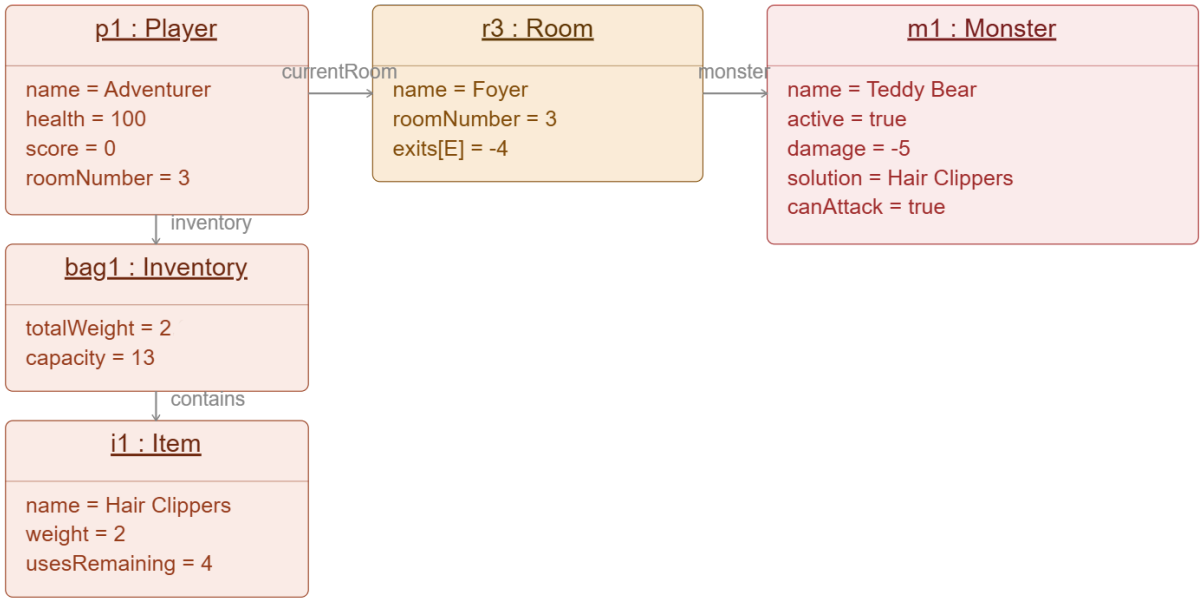
Snapshot: The "Monster Blocking" State

Object Instance	Class	Attributes/State
p1	Player	name="Adventurer", health=100, score=0, roomNumber=3
bag1	Inventory	totalWeight=2, capacity=13
r3	Room	name="Foyer", roomNumber=3, exits[E]=-4
m1	Monster	name="Teddy Bear", active=true, damage=-5, solution="Hair Clippers", canAttack=true
i1	Item	name="Hair Clippers", weight=2, usesRemaining=4, value=5

Logics:

- p1 is in r3 (currentRoom)
- p1 holds bag1 (Inventory)
- bag1 contains i1 (Hair Clippers)
- m1 is in r3 (monster blocks east exit)
- r3.exits[E] = -4 (blocked by m1)
- p1 attempts move E → blocked, m1 attacks → p1.health = 95
- p1 issues U Hair Clippers → i1.usesRemaining = 3
- i1 matches m1.solution → m1.active = false
- r3.exits[E] flips to 4 (unlocked)
- p1.score = 200

Snapshot: The "Monster Blocking" State



Scenario 2: Player Manages Inventory Weight to Progress

Context: The Player is in the Giant Plate room and wants to take the Carrot, but their inventory is too heavy. They must drop the Lamp first before they can pick up the Carrot.

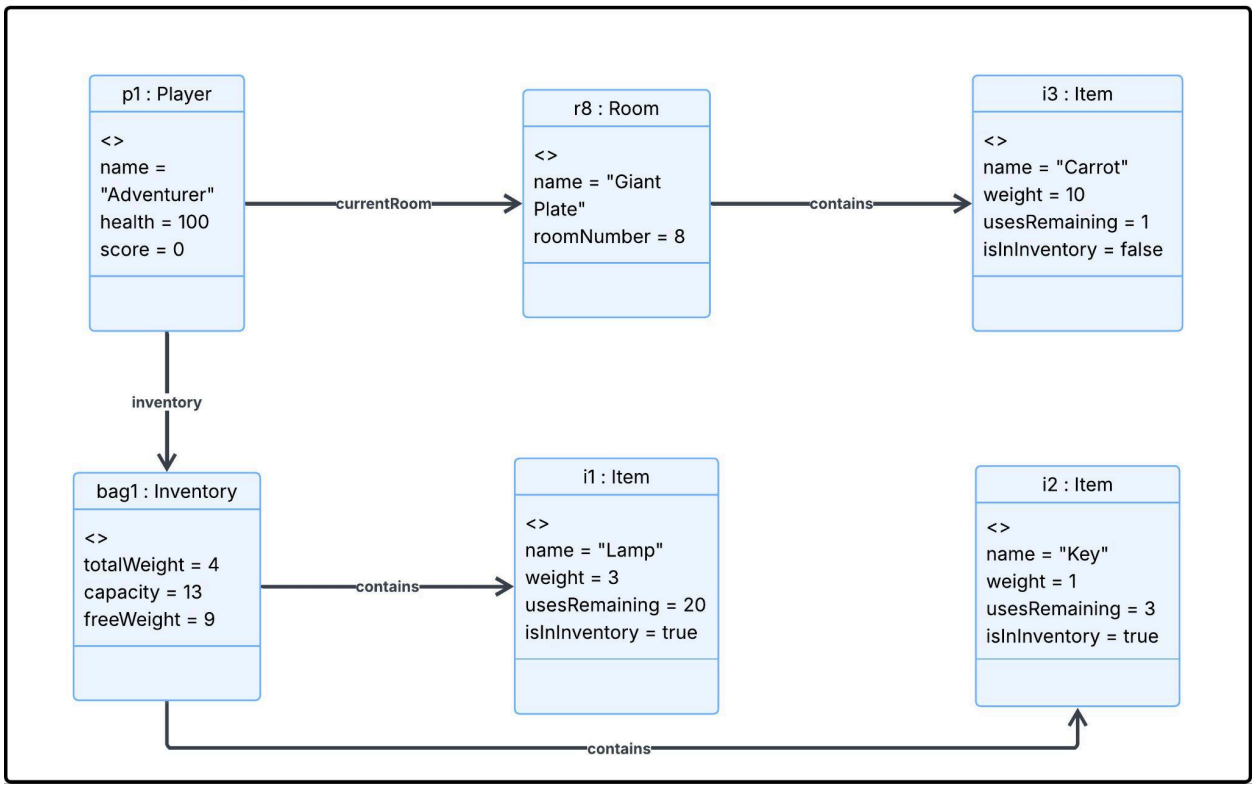
Snapshot: The "Overweight" State

Object Instance	Class	Attributes/State
p1	Player	name="Adventurer", health=100, score=0, roomNumber=8
bag1	Inventory	totalWeight=4, capacity=13, freeWeight=9
r8	Room	name="Giant Plate", roomNumber=8, items=[carrot]
i1	Item	name="Lamp", weight=3, usesRemaining=20, value=100, isInInventory=true
i2	Item	name="Key", weight=1, usesRemaining=3, value=5, isInInventory=true
i3	Item	name="Carrot", weight=10, usesRemaining=1, value=5, isInInventory=false

Logics:

- p1 is in r8 (current location)
- p1 holds bag1 (Inventory)
- bag1 contains i1 (Lamp, weight=3)
- bag1 contains i2 (Key, weight=1)
- i3 is in r8(room contains Carrot)
- p1 attempts T Carrot → **fails**
- p1 issues D Lamp → bag1.totalWeight = 1, i1 moves to r8
- p1 attempts T Carrot → **succeeds**
- bag1.totalWeight = 11 after successful take

Snapshot: The "Overweight" State



4. UML Sequence Diagrams

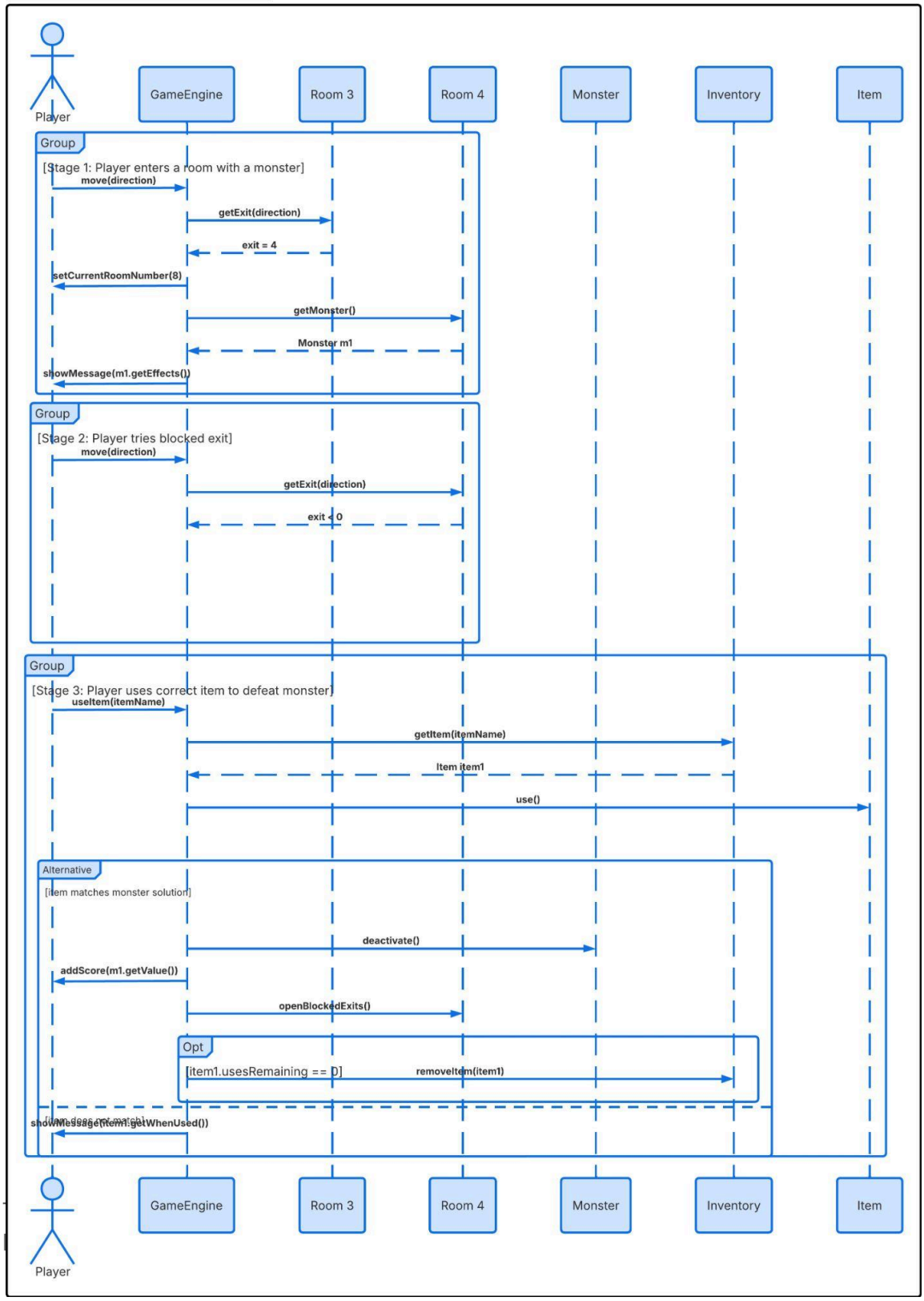
Scenario 1: Player Uses an Item to Defeat a Monster

Logics:

```
// Stage 1: Go into a room has a monster
p1 → GameEngine : move(direction)
GameEngine → r : getExit(direction)
r → GameEngine : exit > 0
GameEngine → p1 : setCurrentRoomNumber(exit)
GameEngine → r_new : getMonster()
r_new → GameEngine : monster(m)
GameEngine → p1 : showMessage(m.getEffects())

// Stage 2: Player tries to go to blocked exit
p1 → GameEngine : move(direction)
GameEngine → r : getExit(direction)
r → GameEngine : exit < 0

opt [m.isActive() AND m.canAttack()]
   GameEngine → p1 : takeDamage(m.getDamage())
   GameEngine → p1 : showMessage(attackMessage)
end

// Stage 3: Player defeats the monster by using item
p1 → GameEngine : useItem(itemName)
GameEngine → inventory : getItem(itemName)
inventory → GameEngine : item

GameEngine → item : use()
GameEngine → r : getMonster()
r → GameEngine : monster(m)

alt [itemName matches m.getSolution()]
   GameEngine → m : deactivate()
   GameEngine → p1 : addScore(m.getValue())
   GameEngine → r : openBlockedExits()

  opt [item.usesRemaining == 0]
      GameEngine → inventory : removeItem(item)
  end
```

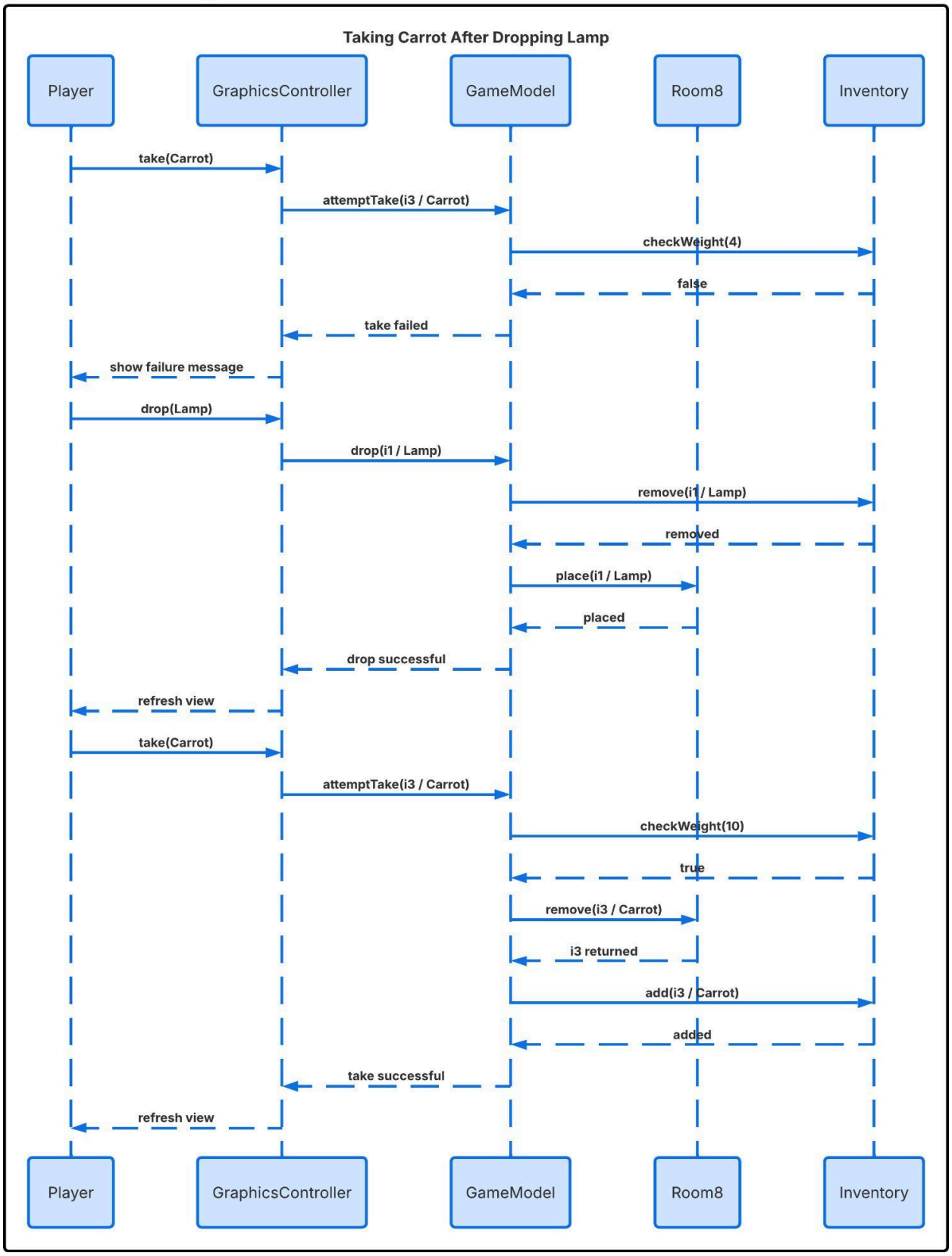
else [no match]
  GameEngine → p1 : showMessage(item.getWhenUsed())
end

Horizontal Sequence Diagram: Player vs Monster Encounter



Scenario 2: Player Manages Inventory Weight to Progress

Logics:

```
// Stage 1: Player tries to get item(failed)
p1 → GameEngine : takeItem(itemName)
GameEngine → r : getItems()
r → GameEngine : itemList
GameEngine → inventory : canAdd(item)
inventory → GameEngine : false

alt [exceeds weight limit]
   GameEngine → p1 : showMessage(“Overweight")
end

// Stage 2: Player drops items
p1 → GameEngine : dropItem(itemName)
GameEngine → inventory : getItem(itemName)
inventory → GameEngine : item
GameEngine → inventory : removeItem(itemName)
GameEngine → r : addItem(item)
GameEngine → p1 : showMessage("You drop item")

// Stage 3: Player get the item
p1 → GameEngine : takeItem(itemName)
GameEngine → r : getItems()
r → GameEngine : itemList
GameEngine → inventory : canAdd(item)
inventory → GameEngine : true
GameEngine → r : removeItem(item)
GameEngine → inventory : addItem(item)
GameEngine → p1 : showMessage("You pick up item")
```

Taking Carrot After Dropping Lamp